

# Test Task: Mini User and Project Management API

## Objective

Implement a small REST API using:

- FastAPI
- Pydantic
- SQLAlchemy or SQLModel
- A relational database

No user interface is required.

The complete application must start locally with a single command:

`docker-compose up`

No additional setup or manual commands should be necessary.

## Domain Model

Implement the following entities:

### User

A user should include at least:

- ID
- Name
- Email

The email address must be unique.

### Project

A project should include at least:

- ID
- Name

- Optional description
- Owner reference

## Relationship

A single User can own multiple Projects.

User 1 → Many Projects

Each Project must belong to one existing User.

## API Requirements

### Users

#### Create a User

POST /users

Requirements:

- Create a new user.
- Validate the request payload.
- Ensure that the email address is unique.
- Return an appropriate error when the email already exists.

#### Retrieve a User

GET /users/{id}

Requirements:

- Retrieve a user by ID.
- Return **404 Not Found** when the user does not exist.

#### List Users

GET /users

Requirements:

- Return a paginated list of users.
- Support the following query parameters:
  - **limit**

- `offset`

Example:

GET /users?limit=20&offset=0

### Delete a User

DELETE /users/{id}

Requirements:

- Delete a user by ID.
- Return `404 Not Found` when the user does not exist.
- Correctly handle projects associated with the deleted user.
- The selected behavior, such as cascading project deletion, should be clearly implemented and documented.

## Projects

### Create a Project

POST /projects

Requirements:

- Create a project for an existing user.
- Validate that the specified owner exists.
- Return an appropriate error when the user does not exist.

### Retrieve a Project

GET /projects/{id}

Requirements:

- Retrieve a project by ID.
- Return `404 Not Found` when the project does not exist.

### List a User's Projects

GET /users/{id}/projects

Requirements:

- Return all projects owned by the specified user.
- Return **404 Not Found** when the user does not exist.

## Docker and Local Execution

The project must run using only:

`docker-compose up`

The Docker Compose configuration must:

- Start the API service.
- Start the database service.
- Expose the API at **`http://localhost:8000`**.
- Automatically create or apply the database schema.
- Configure all required service dependencies and environment variables.
- Require no manual database initialization.

The generated FastAPI documentation should be available at:

`http://localhost:8000/docs`

## Expected Project Structure

A clear separation of responsibilities is expected. For example:

```
app/
├── main.py
├── api/
│   ├── users.py
│   └── projects.py
├── models/
│   ├── user.py
│   └── project.py
├── schemas/
│   ├── user.py
│   └── project.py
├── services/
│   ├── users.py
│   └── projects.py
├── database.py
└── dependencies.py
```

```
tests/
├── test_users.py
└── test_projects.py
```

```
Dockerfile
docker-compose.yml
requirements.txt
README.md
```

This structure is only a recommendation. Alternative structures are acceptable when they remain clean and easy to understand.

## Error Handling

Use appropriate HTTP status codes, including:

- **201 Created** for successful resource creation
- **200 OK** for successful retrieval
- **204 No Content** for successful deletion
- **404 Not Found** when a resource does not exist
- **409 Conflict** when attempting to create a user with an existing email
- **422 Unprocessable Entity** for request validation errors

## Evaluation Criteria

### Core Requirements

The following will be evaluated as must-have requirements:

- Correct use of FastAPI.
- Clean request and response models using Pydantic.
- Correct ORM models and relationships.
- Email uniqueness enforcement.
- Proper validation and error handling.
- Working pagination.
- Correct handling of user deletion and related projects.
- Docker Compose working out of the box.
- Automatic database schema creation.
- Readable, maintainable code.
- Clear separation of concerns.

## Optional Bonus Points

Additional credit may be given for:

- Dependency injection for database sessions.
- Async SQLAlchemy or SQLModel usage.
- Automated tests using pytest.
- OpenAPI tags, endpoint descriptions, and response documentation.
- Database migrations using Alembic.
- Health-check endpoint.
- Type annotations throughout the codebase.
- Linting or formatting configuration.

## Deliverables

Please provide:

- Complete source code.
- `Dockerfile`.
- `docker-compose.yml`.
- Dependency configuration such as `requirements.txt` or `pyproject.toml`.
- A short `README.md` containing:
  - How to start the application.
  - API documentation location.
  - Main architectural decisions.
  - User-deletion behavior for associated projects.
  - Instructions for running tests, when tests are included.

The reviewer should be able to clone the repository and start the entire application using:

```
docker-compose up
```